

Problem Description:

- You are tasked with creating and managing a GitHub repository that will hold a simple Python program that prints "Hello, World!" You will simulate a full version control and collaboration workflow by completing the following steps:

Step 1: Setting Up Git and GitHub

- **Create a GitHub repository:**
 - Name your repository GitHub-Intro.
 - Add a short description in the repository settings.
 - Initialize the repository with a README.md file and a .gitignore file (choose Python from the templates).
 - Take a screenshot of your repository's main page showing the README file.
- **Configure Git on your local machine:**
 - Use the command line to configure Git with your name and email, and take a screenshot of your terminal showing how you configure it.

Step 2: Cloning and Working with the Repository

- **Clone the repository:**
 - Take a screenshot showing the git clone command and the local folder created.
- **Add a Python file:**
 - Inside the local repository folder, create a Python file named hello.py.
 - Write a simple Python program that prints "Hello, World!" in the file.
 - Save the file, and use the command line to add and commit the file:
 - Take a screenshot of your terminal showing the git add and git commit commands.
- **Push the changes to GitHub:**
 - Push the changes from your local machine to your GitHub repository.
 - Take a screenshot showing the successful push and the updated GitHub repository with the hello.py file.

Step 3: Creating and Merging Branches

- **Create a new branch:**
 - Create a new branch called feature-branch for adding a new feature to the program.
 - In this branch, modify hello.py to take a user input and print their name.
 - Add and commit the changes.
 - Take a screenshot of the git branch, git add, and git commit commands.
- **Push the branch to GitHub:**
 - Push the feature-branch to GitHub.
 - Create a **pull request** on GitHub to merge feature-branch into main.
 - Review the pull request, add comments if necessary, and merge it into the main branch.
 - Take screenshots showing the pull request creation, review, and merge process.

Step 4: Collaborating on GitHub

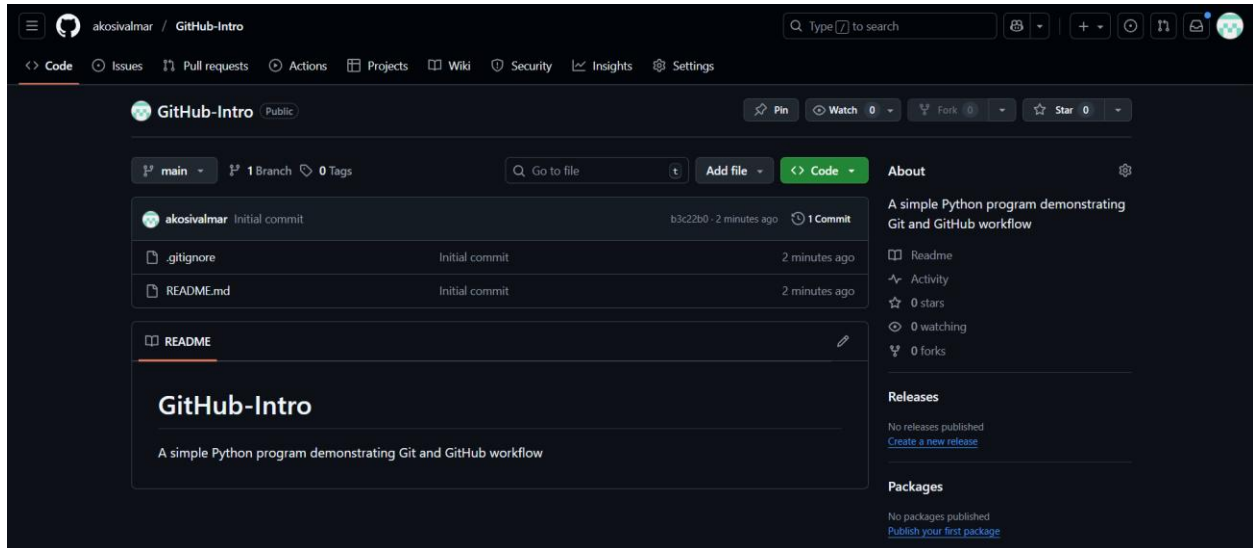
- **Invite a collaborator:**
 - Invite a classmate to collaborate on your repository using GitHub's "Collaborators" feature.
 - Ask your collaborator to clone the repository, create a new branch (fix-branch), and make a minor change to README.md (e.g., fixing a typo or adding a description).
 - Your collaborator should push their branch and create a pull request.
 - Review their pull request and merge it into the main branch.
 - Take a screenshot of the collaborator's pull request and the merge process.

Step 5: Pulling Changes from GitHub

- **Pull the latest changes:**
 - After merging the collaborator's changes, use the git pull command to pull the latest changes to your local repository.

- Take a screenshot showing the git pull command and the updated local repository.

Create a GitHub repository:



Explanation: Creating a repository on GitHub establishes a remote location to store your code. The README.md file serves as the front page of your repository, providing project information.

Configure Git on your local machine:

```

ACER@LAPTOP-SIG8DRPD MINGW64 ~
$ git config --global user.name "akosivalmar"

ACER@LAPTOP-SIG8DRPD MINGW64 ~
$ git config --global user.mail "chrystiankeith@gmail.com"

ACER@LAPTOP-SIG8DRPD MINGW64 ~
$ git config --global --list
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
filter.lfs.clean=git-lfs clean -- %f
user.name=akosivalmar
user.email=chrystiankeith@gmail.com
user.mail=chrystiankeith@gmail.com

ACER@LAPTOP-SIG8DRPD MINGW64 ~
$

```

Explanation: Git configuration sets your identity for all commits. The --global flag applies these settings to all repositories on your machine. Your name and email will appear in the commit history, helping others identify who made specific changes.

Clone the repository:

```

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents
$ cd ~/Documents

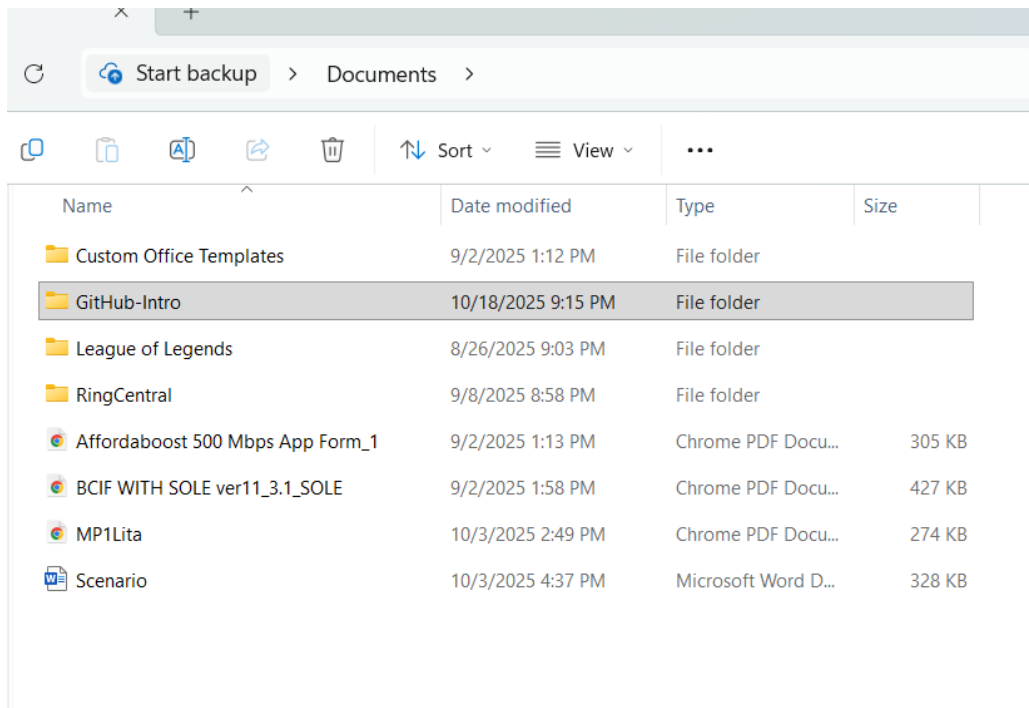
ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents
$ git clone https://github.com/akosivalmar/GitHub-Intro.git
Cloning into 'GitHub-Intro'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents
$ cd GitHub-Intro

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ ls -la
total 17
drwxr-xr-x 1 ACER 197121  0 Oct 18 21:15 ./
drwxr-xr-x 1 ACER 197121  0 Oct 18 21:15 ../
drwxr-xr-x 1 ACER 197121  0 Oct 18 21:15 .git/
-rw-r--r-- 1 ACER 197121 4895 Oct 18 21:15 .gitignore
-rw-r--r-- 1 ACER 197121  79 Oct 18 21:15 README.md

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ |

```



Explanation: Cloning creates a local copy of your GitHub repository on your computer. This copy includes all files, commit history, and branches. The `ls -la` command shows all files including hidden ones like `.git` and `.gitignore`.

Add a Python file:

```
ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ touch hello.py

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ nano hello.py

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.py

nothing added to commit but untracked files present (use "git add" to track)

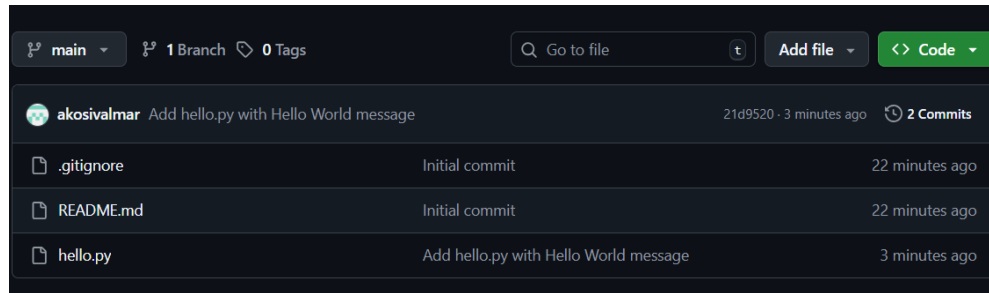
ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ git add hello.py

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ git commit -m "Add hello.py with Hello World message"
[main 21d9520] Add hello.py with Hello World message
 1 file changed, 1 insertion(+)
 create mode 100644 hello.py

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ git log --oneline
21d9520 (HEAD -> main) Add hello.py with Hello World message
b3c22b0 (origin/main, origin/HEAD) Initial commit

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
```

Push the changes to GitHub:



```
ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ git push origin main
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 361 bytes | 361.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/akosivalmar/GitHub-Intro.git
   b3c22b0..21d9520  main -> main
ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$
```

Explanation: git push uploads your local commits to GitHub's remote repository. The origin is the default name for your remote repository, and main (or master) is the branch name. After pushing, your changes are visible on GitHub and accessible to collaborators.

Create a new branch:

```
MINGW64:/c/Users/ACER/Documents/GitHub-Intro
diff --git a/hello.py b/hello.py
index 7df869a..48ab743 100644
--- a/hello.py
+++ b/hello.py
@@ -1,2 @@
-print("Hello, world!")
+name = input("Enter your name: ")
+print(f"Hello, {name}!")
\ No newline at end of file

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (feature-branch)
$ git status
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.py

no changes added to commit (use "git add" and/or "git commit -a")

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (feature-branch)
$ git add hello.py

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (feature-branch)
$ git commit -m "Add user input feature to hello.py"
[feature-branch d8f23fc] Add user input feature to hello.py
1 file changed, 2 insertions(+), 1 deletion(-)

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (feature-branch)
$ git branch -v
* feature-branch d8f23fc Add user input feature to hello.py
  main          21d9520 Add hello.py with Hello World message

ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (feature-branch)
$
```

Explanation: Branches allow you to develop features independently without affecting the main codebase. The checkout -b command creates a new branch and switches to it simultaneously.

Push the branch to GitHub:

Pull request creation page

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: **main** ← compare: **feature-branch** ✓ **Able to merge.** These branches can be automatically merged.

Add user input feature #1 [View pull request](#)

This PR adds functionality to accept user input and print a personalized greeting

1 commit 1 file changed R 1 contributor

Commits on Oct 18, 2025

Add user input feature to hello.py [d8f23fc](#) <>

akosivalmar committed 6 minutes ago

Showing 1 changed file with 2 additions and 1 deletion. [Split](#) [Unified](#)

```
@@ -1,2 @@
1 - print("Hello, World!")
+ name = input("Enter your name: ")
2 + print(f"Hello, {name}!")
```

Explanation: Pushing the branch makes it available on GitHub for review. The -u flag sets up tracking, so future pushes and pulls on this branch don't require specifying the remote and branch name.

Pull request review/comments

Add user input feature #1

[Open](#) akosivalmar wants to merge 1 commit into **main** from **feature-branch**

Conversation 0 Commits 1 Checks 0 Files changed 1 +2 -1

akosivalmar commented 3 minutes ago [Owner](#) ...

This PR adds functionality to accept user input and print a personalized greeting

[Add user input feature to hello.py](#) [d8f23fc](#)

No conflicts with base branch
Merging can be performed automatically.

[Merge pull request](#) You can also merge this with the command line. [View command line instructions](#).

Add a comment

Write Preview H B I `<> @ 🔗 📌 📋 📎 📧 📎 📎`

Add your comment here...

Reviews
No reviews
Still in progress? [Convert to draft](#)

Assignees
No one—assign yourself

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.

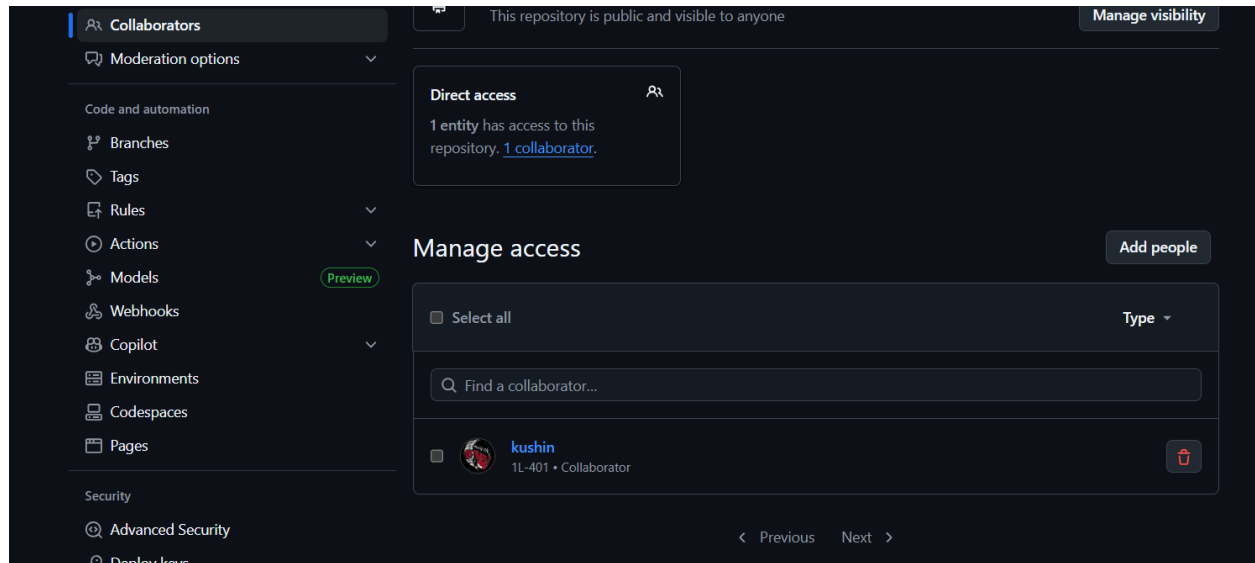
Successful merge confirmation

Pull request successfully merged and closed

You're all set — the **feature-branch** branch can be safely deleted.

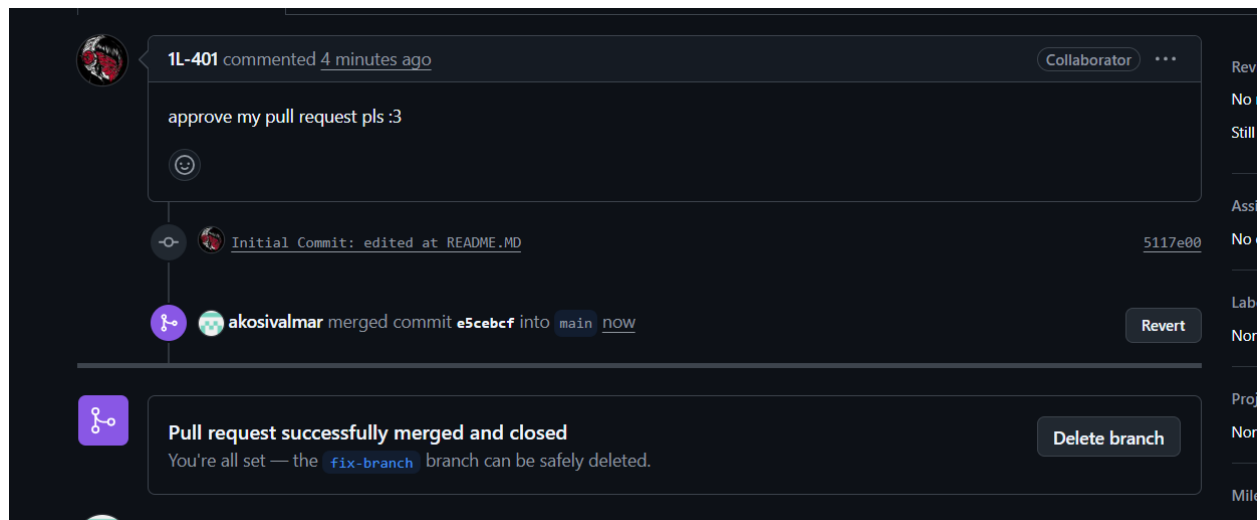
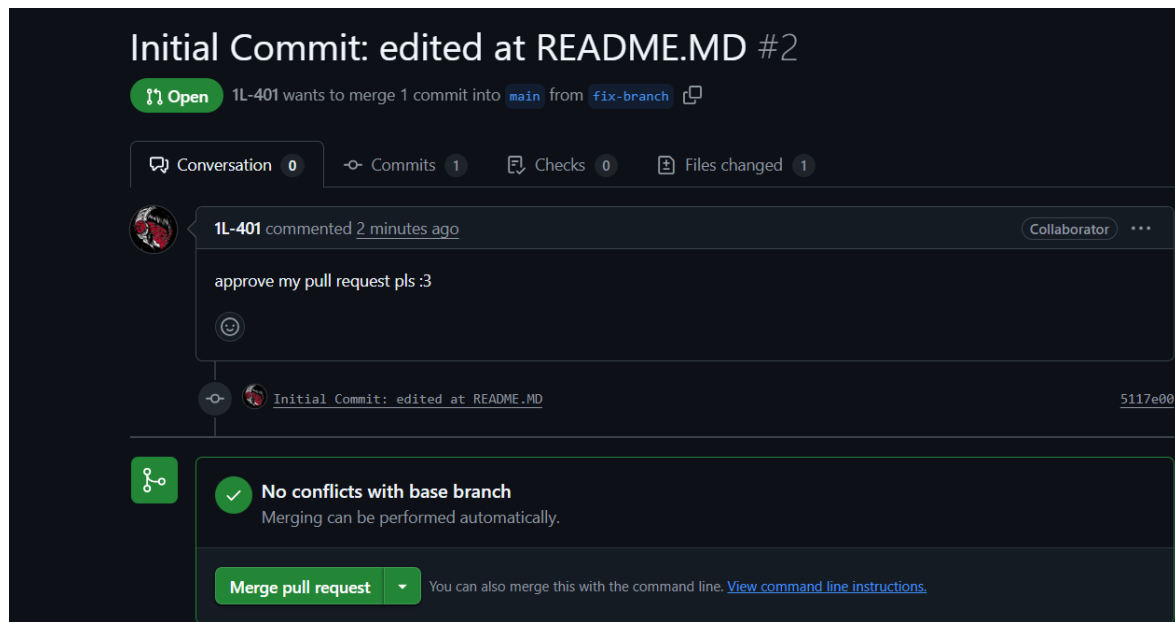
[Delete branch](#)

Explanation: Pull requests are a way to propose changes and facilitate code review. They allow team members to discuss modifications before merging them into the main branch. The PR interface shows all changes, allows inline comments, and provides a clear history of discussions. Merging integrates the feature branch changes into the main branch.



Explanation: Adding collaborators grants them permission to push directly to your repository. They can create branches, submit pull requests, and review code. This is essential for team projects and demonstrates real-world collaborative development.

Take a screenshot of the collaborator's pull request and the merge process.



Explanation: Code review is crucial in collaborative development. Reviewing PRs ensures code quality, catches bugs, and facilitates knowledge sharing. Constructive feedback helps improve the codebase and team skills.

Git Pull command:

```
ACER@LAPTOP-SIG8DRPD MINGW64 ~/Documents/GitHub-Intro (main)
$ git pull origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 2.17 KiB | 247.00 KiB/s, done.
From https://github.com/akosivalmar/GitHub-Intro
* branch      main      -> FETCH_HEAD
  21d9520..e5cebcf main    -> origin/main
Updating 21d9520..e5cebcf
Fast-forward
 README.md | 3 +++
 hello.py  | 3 ++-
 2 files changed, 5 insertions(+), 1 deletion(-)
```

Explanation: git pull fetches changes from GitHub and merges them into your local branch. This keeps your local repository synchronized with the remote. It's essential to pull regularly when working in a team to avoid conflicts and stay up-to-date with others' changes.